
Advanced Higher Computing Science

Software Design and Development using Python



Booklet 1 – Object Oriented Programming

Section 1 – Object Oriented Programming	2
Classes, Properties and Methods	2
Description	2
Designing Classes, Properties and Constructor Method	3
Implementing Classes, Properties and Constructor Method in Python 3	4
Testing Classes, Properties and Constructor Method in Python 3	5
Designing Methods	6
Implementing Methods in Python 3	7
Testing Methods in Python 3	7
Classes, Properties and Methods - Exercise	8
Encapsulation	9
Description	9
Designing Encapsulation	10
Implementing Encapsulation in Python 3	10
Testing Encapsulation in Python 3	11
Encapsulation – Exercise	12
Inheritance	13
Description	13
Designing Inheritance	13
Implementing Inheritance in Python 3	15
Testing Inheritance in Python 3	16
Inheritance – Exercise	17

Section 1 – Object Oriented Programming

Classes, Properties and Methods

Description

Object-oriented programming (OOP) is a computer programming model that organizes software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique properties and behaviour.

The organization of an object-oriented program also makes the method beneficial to collaborative development, where projects are divided into groups. Additional benefits of OOP include code reusability, scalability and efficiency.¹

Definitions:

Objects are instances of a class created with specifically defined data. Objects can correspond to real-world objects or an abstract entity. When class is defined initially, the description is the only object that is defined.

Classes are user-defined data types that act as the blueprint for individual objects, properties, and methods.

Properties are defined in the class template and represent the state of an object. Objects will have data stored in the properties field. Class properties belong to the class itself.

Methods are functions that are defined inside a class that describe the behaviours of an object. Each method contained in class definitions starts with a reference to an instance object. Additionally, the subroutines contained in an object are called instance methods. Programmers use methods for reusability or keeping functionality encapsulated (see later section on encapsulation) inside one object at a time.

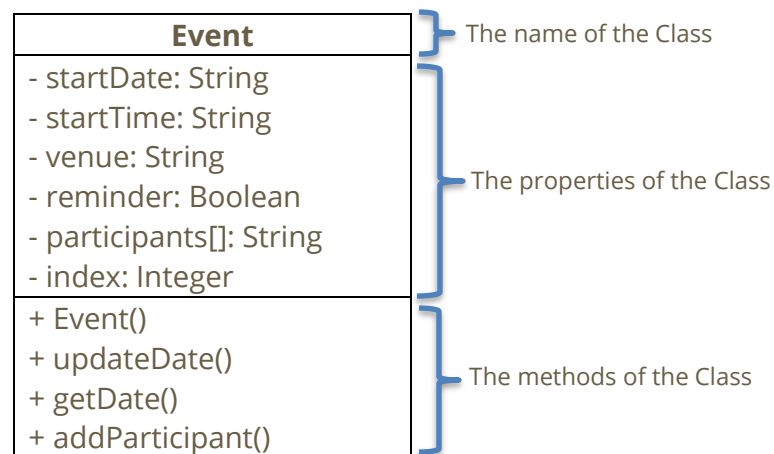
The **Constructor Method** is a special method that all classes must have as it defines the initial, or starting, state of any Objects created from the class.

Instantiation is the creation of a specific object from the Class, using the Constructor method.

¹ <https://searchapparchitecture.techtarget.com/definition/object-oriented-programming-OOP>

Designing Classes, Properties and Constructor Method

Unified Modelling Language (UML), as well as developing a standardised notation for Use Case Diagrams (covered in Analysis), also provides a standardised notation for designing Classes called **Class Diagrams**. An example of a Class called **Event** (adapted from the 2019 SQA Question Paper²) is shown below.



In the example above, we can see 6 properties and 4 methods for the **Event** Class.

Note that the properties all have a type of data defined. In particular, note that the property *participants[]* is an array of String, showing that an objects' properties are not limited to single pieces of data and could be an array of a data type, an array of records of even an array of objects!

The first method, *Event()*, is the Constructor Method for Objects of this class.

In this Class, the properties *startDate*, *startTime*, *venue* and *reminder* will need to be passed in, as parameters, to the Object when it is instantiated.

The *participants[]* and *index* properties will have default values set within the class when an Object is instantiated.

² https://www.sqa.org.uk/pastpapers/papers/papers/2019/NAH_Computing-Science_QP_2019.pdf

Using SQA Reference Language, the following would provide the required properties and Constructor method for creating the properties for the **Event** class.

```

Line 1    CLASS Event IS { STRING date, STRING startTime, STRING venue, BOOLEAN reminder,
            ARRAY OF STRING participants, INTEGER index }

Line 2        METHODS
Line 3        CONSTRUCTOR (STRING startDate, STRING startTime, STRING venue, BOOLEAN
            reminder )
Line 4            DECLARE THIS.startDate INITIALLY startDate
Line 5            DECLARE THIS.startTime INITIALLY startTime
Line 6            DECLARE THIS.venue INITIALLY venue
Line 7            DECLARE THIS.alert INITIALLY alert
Line 8            DECLARE THIS.participants INITIALLY [NULL] * 20
Line 9            DECLARE THIS.index INITIALLY 0
Line 10        END CONSTRUCTOR

Line 11    END CLASS

```

Implementing Classes, Properties and Constructor Method in Python 3

The following Python 3 code is used to create the Constructor method for the **Event** class.

```

class Event:
    def __init__(self, startDate, startTime, venue, reminder):
        self.date = startDate      #String
        self.startTime = startTime #String
        self.venue = venue         #String
        self.reminder = reminder   #Boolean
        self.participants = [""]*20 #Array of String
        self.index = 0             #Integer

```

Program 1 - defining a class

Note the following when implementing the Constructor Method in Python 3:

- **Class** has a small 'c'
- **__init__** has 2 underscores at the start and end, with no spaces (this is called a double underscore, or *dunder* for short).
- Python has no defined way of declaring a data type, so internal commentary is used to be specific about the data types/ structures being used.
- Running the code above will not have any visible effect at the current time, as all it does is create the **Event** Class – no Objects have been created, and no methods have been added to the class to provide behaviours.

Testing Classes, Properties and Constructor Method in Python 3

To test the Constructor method works as expected, an Object will need to be created (instantiated) using the Class definition, and a simple `print()` used to ensure that data has been entered correctly.

Adding the following code after the class definition (with no indentation) will create an object called `item1` of the class `Event` and populate it with the given data.

```
item1 = Event("13/04/18", "0900", "Main Office", True)
print (item1.venue)
```

Program 2 – instantiate and test

The `print` line uses *dot notation* to read the details from the `venue` property from the Object `item1`.

Remember, *participants()* and *index* are defined within the class so will have their values set to the default given in the Constructor method.

Try changing the property in the `print` line to make sure all 6 properties have been correctly assigned.

Designing Methods

As in Higher Computing Science, a method is a module that can be given to a class. These can be functions or procedures.

In the **Events** Class Diagram, 4 methods are identified (Event(), updateDate(), getDate() and addParticipant()). The Event() method has already been created, as it is the Constructor method for the class.

Using SQA Reference Language, the following would provide the required remaining methods for creating the **Event** class.

```

Line 1    CLASS Event IS { STRING startDate, STRING startTime, STRING venue, BOOLEAN reminder,
            ARRAY OF STRING participants, INTEGER index }

Line 2    METHODS
Line 3    CONSTRUCTOR (STRING startDate, STRING startTime, STRING venue, BOOLEAN
            reminder )
Line 4        DECLARE THIS.startDate INITIALLY date
Line 5        DECLARE THIS.startTime INITIALLY startTime
Line 6        DECLARE THIS.venue INITIALLY venue
Line 7        DECLARE THIS.alert INITIALLY alert
Line 8        DECLARE THIS.participants INITIALLY [NULL] * 20
Line 9        DECLARE THIS.index INITIALLY 0
Line 10    END CONSTRUCTOR

Line 11    PROCEDURE updateDate (STRING eventDate)
Line 12        SET THIS.startDate TO eventDate
Line 13    END PROCEDURE

Line 14    FUNCTION getDate() RETURNS STRING
Line 15        RETURN THIS.startDate
Line 16    END FUNCTION

Line 17    PROCEDURE addParticipant(STRING name)
Line 18        SET THIS.participants[index] TO name
Line 19        SET index TO index + 1
Line 20    END PROCEDURE

Line 21    END CLASS

```

Implementing Methods in Python 3

The following Python 3 code is used to create the remaining methods for the **Event** class.

```
class Event:
    def __init__(self, startDate, startTime, venue, reminder):
        self.startDate = startDate #String
        self.startTime = startTime #String
        self.venue = venue #String
        self.reminder = reminder #Boolean
        self.participants = [""]*20 #Array of String
        self.index = 0 #Integer

    def updateDate(self, eventDate):
        self.startDate = eventDate

    def getDate(self):
        return self.startDate

    def addParticipant(self, name):
        self.participants[self.index] = name
        self.index = self.index + 1
```

Program 3 – creating methods in a class

Note the following when implementing the Methods in Python 3:

- Each **def** is indented to the same level as the **def __init__** Constructor method.
- Each **def** requires the **self** parameter at the start
- To refer to the current position in the **participants** array, there is a need to use **self.index**

Quite often, methods are referred to as *getter* or *setter* methods, depending on whether they are being used to *get* information from an Object (like the **getDate()** method or being used to *set* or update information in an Object (such as the **updateDate()** method). Although not part of the Advanced Higher Computing Science course, it is often useful to think about methods in this way.

Testing Methods in Python 3

To make sure the methods work, they each need called in turn. It is important, when testing, to show the initial state of the object's properties followed by the impact of calling the method.


```

#Test for the CONSTRUCTOR method
item1 = Event("13/04/18", "0900", "Main Office", True)
print (item1.venue)

#Test the updateDate() method by showing original date
#then showing the new date
print (item1.startDate, "is the original date")
item1.updateDate("13/04/22")
print (item1.startDate, "is the updated date")

#Test the getDate() method by printing the returned value
print (item1.getDate(), "is the date from the getDate() method")

#Test the addParticipant() method is allocating names and incrementing index
print (item1.index)
print (item1.participants)
item1.addParticipant ("Erica Knowles")
item1.addParticipant ("Norman Osborne")
item1.addParticipant ("Fred Savage")
print (item1.index)
print (item1.participants)

```

Program 4 – testing methods in a class

The above code can be added to ensure that all methods work as expected.

Classes, Properties and Methods - Exercise

For the following scenarios, provide evidence of design (UML Class Diagram and Pseudocode), implementation and testing.

1. A Vehicle class with properties number of wheels, colour, type of fuel, seating capacity, roof or not, maximum velocity and a passenger list. Methods will be needed to paint the vehicle, find the vehicles seating capacity, and change the number of wheels.
2. An Animal class for a zoo with properties date of birth, number of feet, colour, a list of things it eats, its location, if it can fly or not, what noise it makes. Methods will be needed to change the colour, change its location, and to get the number of feet it has.
3. A Computer class for a school with suitable properties and methods.

Encapsulation

Description

Encapsulation refers to a mechanism of restricting the direct access to some components of an object, such that users cannot access state values for all the properties of a particular object.

Definitions:

Encapsulation refers to the bundling of data with the methods that operate on that data, or the restricting of direct access to some of an object's components. Encapsulation is used to hide the values or state of a structured data object inside a class, preventing direct access to them by clients in a way that could expose hidden implementation details or violate state invariance maintained by the methods.³

Public Properties are able to be viewed and modified directly by referring to them using dot notation.

Private Properties make it impossible for users of a class to directly view or modify their values. The designer of the class then provides methods to control the ways in which these properties can be manipulated.

Public methods are able to be used by any other class method within the program to manipulate properties.

Private methods are only available to methods within the class itself – no other classes within a program can access private methods.

³ [https://en.wikipedia.org/wiki/Encapsulation_\(computer_programming\)](https://en.wikipedia.org/wiki/Encapsulation_(computer_programming))

Designing Encapsulation

The UML Class Diagram met earlier shows the following.

Event
- startDate: String - startTime: String - venue: String - reminder: Boolean - participants[]: String - index: Integer
+ Event() + updateDate() + getDate() + addParticipant()

Each of the properties is annotated with a -. This indicates these properties are private and should not be directly accessible to either view or modify the values they contain. Instead, the developer should provide either getter or setter methods to allow the data in these properties to be viewed or modified.

The methods are all marked with a +. This indicates all methods are available to other classes within the program.

Implementing Encapsulation in Python 3

Python 3 has no specific keywords to define properties or methods as private or public, so a workaround is used instead.

As Python 3 makes all properties and methods public by default, it is important to mark any private properties and methods with a dunder (double underscore). In the example below, the properties have all been marked private, to match the UML Class Diagram.

```

class Event:
    def __init__(self, startDate, startTime, venue, reminder):
        self.__startDate = startDate #String
        self.__startTime = startTime #String
        self.__venue = venue         #String
        self.__reminder = reminder   #Boolean
        self.__participants = [""]*20 #Array of String
        self.__index = 0             #Integer

    def updateDate(self, eventDate):
        self.startDate = eventDate

    def getDate(self):
        return self.startDate

    def addParticipant(self, name):
        self.participants[self.index] = name
        self.index = self.index + 1

```

Program 5 – encapsulation

Note, the use of dunder in `__init__` essentially makes the Constructor method private, which is very sensible!

Testing Encapsulation in Python 3

There is very little testing that can be done to show that a property or method is public.

To ensure that an property is private, dot notation can be used to try to access the property directly from the main program.

```

#Test for the CONSTRUCTOR method
item1 = Event("13/04/18", "0900", "Main Office", True)
print (item1.venue)

```

Program 6 – testing encapsulation

Running this will give the error `AttributeError: 'Event' object has no property 'venue'`, showing that it cannot be directly accessed.

Changing the print line to refer to `item1.__venue` gives a similar error. Try it for yourself to see.

Encapsulation – Exercise

For the following scenarios, provide evidence of design (UML Class Diagram and Pseudocode), implementation and testing. **All properties should be private and all methods should be public.**

1. A Puppy class with properties size, breed, age, sold mother, and father. Methods will be needed to change the age and change if it is sold or not.
2. A Computer class for a school with suitable properties and methods.

Inheritance

Description

Inheritance is the procedure in which one class inherits the attributes and methods of another class. The class whose properties and methods are inherited is known as the Parent class. The class that inherits the properties from the parent class is the Child class. For Advanced Higher Computing Science, the Parent class can be referred to as the Super-class and the Child class is called a subclass.

Definitions:

Inheritance refers to the ability for one class to take existing properties and methods from a Parent class.

Parent class is the class from which all the properties and methods are inherited.

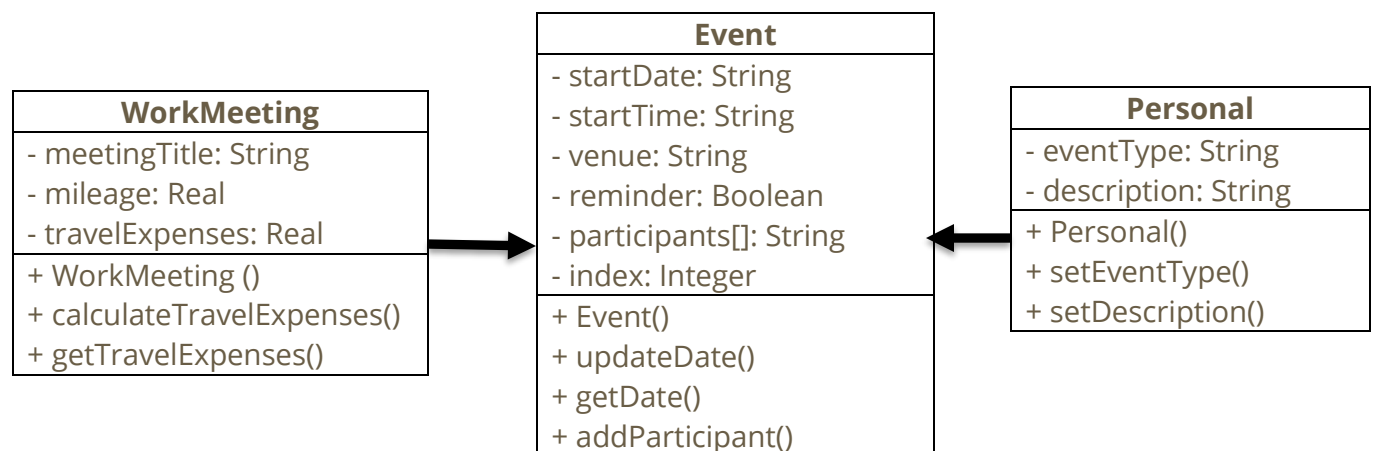
Superclass is the Advanced Higher Computing Science term for a Parent class.

Child class is the class to which all the properties and methods are passed.

Subclass is the Advanced Higher Computing Science term for a Child class.

Designing Inheritance

The UML Class Diagram below shows two new classes inheriting the properties and methods from the Event Superclass/ Parent class.



The new **WorkMeeting** Class inherits all 6 properties and 4 methods from the **Event** Class, and adds its own properties (*meetingTitle*, *mileage* and *travelExpenses*) and methods (*WorkMeeting()*, *calculateTravelExpenses()* and *getTravelExpenses()*). The *WorkMeeting()* method is the Constructor Method for this subclass.

The SQA Reference Language representation for this class would look like the following, where Lines 1-21 are the Class definition of the Event Class:

```

Line 22    CLASS WorkMeeting INHERITS Event WITH { STRING meetingTitle, REAL mileage, REAL
              travelExpenses }

Line 23          METHODS
Line 24          CONSTRUCTOR (STRING meetingTitle, REAL mileage, REAL travelExpenses )
Line 25              DECLARE THIS.meetingTitle INITIALLY meetingTitle
Line 26              DECLARE THIS.mileage INITIALLY mileage
Line 27              DECLARE THIS.travelExpenses INITIALLY 0
Line 28          END CONSTRUCTOR

Line 29          PROCEDURE calculateTravelExpenses ()
Line 30              SET THIS.travelExpenses TO THIS.mileage *1.5
Line 31          END PROCEDURE

Line 32          FUNCTION getTravelExpenses() RETURNS REAL
Line 33              RETURN THIS.travelExpenses
Line 34          END FUNCTION

Line 35    END CLASS

```

Implementing Inheritance in Python 3

The following code gives an example of how to implement inheritance in Python 3:

```
class WorkMeeting(Event):
    def __init__(self, startDate, startTime, venue, reminder, meetingTitle,
mileage ):
        super().__init__(startDate, startTime, venue, reminder)
        self.meetingTitle = meetingTitle      #String
        self.mileage = mileage                 #Real
        self.travelExpenses = 0               #Real

    def calculateTravelExpenses(self):
        self.travelExpenses = self.mileage * 1.5

    def getTravelExpenses(self):
        return self.travelExpenses
```

Program 7 – inheritance

Note the following:

- The class definition shows that WorkMeeting inherits from Event by placing Event in brackets: `class WorkMeeting(Event):`
- The `def __init__` Constructor method features the 4 properties from the superclass Event and the 2 new properties for the WorkMeeting subclass. (travelExpenses is not passed in as a parameter so is not required)
- The `super().__init__` features the 4 properties from the superclass Event. There are other ways of doing this, but this method ensures that the 4 properties are inherited correctly.

Testing Inheritance in Python 3

To test inheritance, first, an object of the subclass is instantiated. A simple print is used to make sure that the data placed in one of the inherited methods is accessible.

```
#Test for the CONSTRUCTOR method (inheriting from Event class)
workMeeting1 = WorkMeeting(startDate = "14/04/21", startTime = "0900", venue =
"Main Office", reminder = True, meetingTitle = "Work from Home", mileage = 7.5)

print ("The date of the meeting was" , workMeeting1.getDate())
```

Program 8 –testing inheritance 1

Note that, in the example above, the instantiation of the object `WorkMeeting1` uses the property names directly. This can be useful if, for example, some of them are not required to be completed.

Note also the use of the inherited method `getDate()` instead of accessing the `startDate` property directly.

The second and third tests are for the two methods in the `WorkMeeting` subclass.

```
#Test for the 2 methods in WorkMeeting class
print ("Before calculation, the travel expenses are £",
workMeeting1.getTravelExpenses())
workMeeting1.calculateTravelExpenses()
print ("After calculation, the travel expenses are £",
workMeeting1.getTravelExpenses())
```

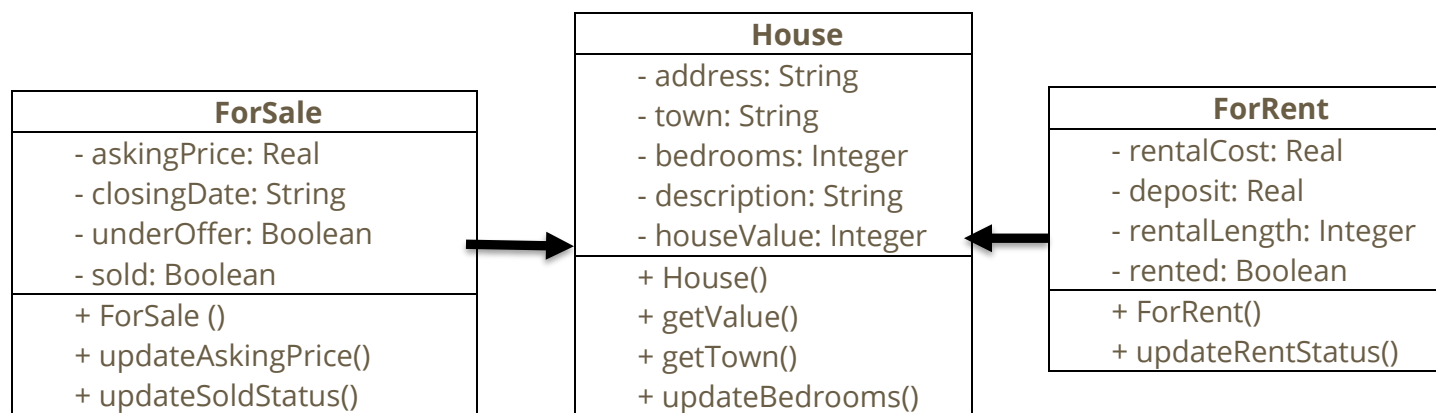
Program 9 – testing inheritance 2

Again, note the use of `print` before and after the call to the `calculateTravelExpenses()` method to check the contents of the property to ensure that the correct change has taken place.

Inheritance – Exercise

1. Using the UML Class Diagram showing **Events**, **WorkMeeting** and **Personal**:
 - a. Create design for the Class definition of the Personal Class, inheriting from the Event Class.
 - b. Implement code in Python to create the Personal Class, inheriting from the Event Class.
 - i. The setEventType() method is used to state what type of personal event is being created (e.g. holiday, doctor appointment, family etc). This should be entered by the user.
 - ii. The setDescription() method is used to allow the user to enter a short piece of text to describe the nature of the event. (e.g. *I am going on holiday on this day and will not be contactable*). This should be entered by the user.
 - c. Provide evidence of suitable testing of the code.

2. Using the UML Class Diagram and description below, design, implement and test the system as shown⁴:



⁴ Scenario adapted from SQA Advanced Higher Computing Science Question Paper 2018:
https://www.sqa.org.uk/pastpapers/papers/papers/2018/NAH_Computing-Science_QP_2018.pdf



Class	Method	Description
ForSale	updateAskingPrice()	Allows the user to enter a new Asking price, validated between 50 000 and 1 000 000
ForSale	updateSoldStatus()	Will check the current status of the sold property and switch it to the opposite value.
House	getValue()	Returns the current value of the house.
House	getTown()	Returns the town the house is located in.
House	updateBedrooms()	Shows the user the current number of bedrooms and then allows them to enter a new value
ForRent	UpdateRentStatus()	Will check the current status of the rented property and switch it to the opposite value.